

Method overriding

Method overriding is the process of implementing super class methods in subclass with same signature.

OOPS353.java

```
class Hello {
    void m1(){
        System.out.println("hello->m1()");
    }
    void m2(){
        System.out.println("hello->m2()");
    }
}
class Hai extends Hello{
    void m2(){
        System.out.println("Hai->m2()");
    }
    void m3(){
        System.out.println("Hai->m3()");
    }
}
```

```
class OOPS353{
    public static void main(String args[]){
        Hai hai=new Hai();
        hai.m1();
        hai.m2();
        hai.m3();
    }
}
```

⇒ Here, Hai class has implemented m2 () method of Hello class with the same signature i.e. m2 () method of Hello class is overridden in Hai class.

⇒ m1 () method of Hello class is not implemented in Hai class but m1 () method of Hello class is inherited to Hai class.

⇒ m3 () method is the newly added method in Hai class.

Why to override the method?

1. According to Object Oriented best practices “Classes are closed for modifications” i.e. if you want to modify the functionality of existing class then you should not disturb existing class.
2. It is always better to write a sub class and provide the required implementations in subclass.
3. Subclass can be written for following purposes.
 - ⇒ To add new functionality.
 - ⇒ To modify existing functionality.
 - ⇒ To inherit existing functionality.

Rules to override the method

1. Subclass method name must be same as super class method name
2. Subclass method parameters (type, order and number) must be same as super class method parameter.
3. Subclass method return type must be same as super class method return type.
Note: If the super class method return type is class type then while overriding the method in subclass you can use same class type or its subclass as return type.(from java5)
4. Subclass method access modifier must be same or higher then super class method access modifier.

Super class	Subclass
public	public
protected	protected, public
default	default, protected, public
private	private, default, protected, public

5. When super class method is instance method then you have to override in sub class as instance only.
6. When super class method is static method then you have to override in sub class as static only.
7. When super class method is throwing some method level checked exception then sub class method can do the following.
 - a. Subclass method can ignore that method level exception.
 - b. Subclass method can throw the same exception.
 - c. Subclass method can throw the exception which is sub class to super class method exception.
 - d. Subclass method cannot throw the exception which is super class to super class method exception.
 - e. Subclass method cannot throw the exception which is non subclass to super class method exception.
 - f. Subclass method can throw any unchecked exception.
8. When super class method is throwing some method level unchecked exception then sub class method can do the following.
 - a) Subclass method can ignore that method level exception.
 - b) Subclass method can throw the same exception.
 - c) Subclass method can throw any other unchecked exception.
 - d) Subclass method cannot throw any checked exception.
9. When you modify the method argument in subclass then it is method overloading not overriding.
10. When your super class method return type is primitive then you have to use exactly same return type while overriding this method in subclass.
11. When your super class method return type is class Type then you can use exactly same return type or covariant type (sub class of super class method return type) while overriding this method in subclass.
12. When the method is overridden in the sub class and if you are using sub class object to invoke the method then implementation from sub class will be invoked.
13. Final method of super class cannot be overridden in sub class.
14. Non final super class can be overridden as final in sub class.
15. Native method of super class can be overridden as non native in sub class.
16. Non native method of super class can be overridden as native in subclass.
17. strictfp method of super class can be overridden as non strictfp in sub class.
18. Non strictfp method of super class can be overridden as strictfp in sub class.
19. Synchronized method of super class can be overridden as non synchronized in sub class.
20. Non synchronized method of super class can be overridden as synchronized in sub class.
21. Private methods of super class will not be visible to sub classes. So whatever method you are writing in sub class that will be treated as new method not overridden method.
22. Access modifier of sub class method must be same or higher the super class method.
23. In the case of method overriding
 - ⇒ When you invoke the method with super class object then method will be called from super class only.
 - ⇒ When you invoke the method with sub class object then method will be called from subclass only.

Programs:

OOPS354.java

```

class OOPS354{
public static void main(String args[]){
    B obj=new B();
    obj.show();
    obj.SHOW();
}
class A{
void show(){
    System.out.println("A->show()");
}
}
class B extends A{
void SHOW(){
    System.out.println("B->SHOW()");
}
}

```

OOPS355.java

```

class OOPS355{
public static void main(String args[]){
    B obj=new B();
    obj.show(99);
    obj.show("sai");
}
class A{
void show(int ab){
    System.out.println("A->show(int)");
}
}
class B extends A{
void show(String ab){
    System.out.println("B->SHOW(String)");
}
}

```

OOPS356.java

```

class OOPS356{
public static void main(String args[]){
    B obj=new B();
    obj.show();
}
class A{
long show(){
return o;
}
}
class B extends A{
int show(){
return o;
}
}

```

OOPS357.java

```

class OOPS357{
public static void main(String args[]){
    Hai hai=new Hai();
    hai.m1();
}
class Hello {
    A m1(){
return new A();
    }
}
class Hai extends Hello{
    B m1(){
return new B();
    }
}
class A{}
class B{}

```

OOPS358.java

```

class OOPS358{
public static void main(String args[]){
    Hai hai=new Hai();
    hai.m1();
}
class Hello {
    A m1(){
return new A();
    }
}
class Hai extends Hello{
    A m1(){
return new A();
    }
}
class A{}
class B extends A{}

```

OOPS359.java

```

class OOPS359{
public static void main(String args[]){
    Hai hai=new Hai();
    hai.m1();
}
class Hello {
    A m1(){
return new A();
    }
}
class Hai extends Hello{
    B m1(){
return new B();
    }
}
class A{}
class B extends A{}

```

OOPS360.java

```

class OOPS360{
    public static void main(String args[]){
        B obj=new B();
        obj.m1();
    }
    class A{
        final void m1(){
        }
    }
    class B extends A{
        void m1(){
        }
    }
}

```

OOPS361.java

```

class OOPS361{
    public static void main(String args[]){
        B obj=new B();
        obj.m1();
    }
    class A{
        void m1(){
        }
    }
    class B extends A{
        final void m1(){
        }
    }
}

```

OOPS362.java

```

class OOPS362{
    public static void main(String args[]){
        B obj=new B();
        obj.m1();
    }
    class A{
        void m1(){
        }
    }
    class B extends A{
        static void m1(){
        }
    }
}

```

OOPS363.java

```

class OOPS363{
    public static void main(String args[]){
        B.m1();
    }
    class A{
        static void m1(){
        }
    }
    class B extends A{
        void m1(){
        }
    }
}

```

OOPS364.java

```

class OOPS364{
    public static void main(String args[]){
        B.m1();
    }
    class A{
        void m1(){
        }
        static void m2(){
        }
    }
    class B extends A{
        void m1(){
        }
        static void m2(){
        }
    }
}

```

OOPS365.java

```

class OOPS365{
    public static void main(String args[]){
        new B().m1();
    }
    class A{
        native void m1();
        void m2(){
        }
    }
    class B extends A{
        void m1(){
        }
        native void m2();
    }
}

```

OOPS366.java

```

class OOPS366{
    public static void main(String args[]){
        new B().m1();
    }
    class A{
        private void m1(){
        }
    }
    class B extends A{
        int m1(){
            return 0;
        }
    }
}

```

OOPS367.java

```

class OOPS367{
    public static void main(String args[]){
        new B().m1();
    }
    class A{
        void m1(){
        }
    }
    class B extends A{
        private void m1(){
        }
    }
}

```

OOPS368.java

```

class OOPS368{
public static void main(String args[]){
new B().m1();
}
class A{
void m1(){ }
void m2(){ }
void m3(){ }
}
class B extends A{
void m1(){ }
protected void m2(){ }
public void m3(){ }
}

```

OOPS369.java

```

class OOPS369{
public static void main(String args[]){
new B().m1();
}
class A{
protected void m1(){ }
protected void m2(){ }
}
class B extends A{
private void m1(){ }
void m2(){ }
}

```

OOPS370.java

```

class OOPS370{
public static void main(String args[]){
new B().m1();
}
class A{
protected void m1(){ }
protected void m2(){ }
}
class B extends A{
protected void m1(){ }
public void m2(){ }
}

```

OOPS371.java

```

class OOPS371{
public static void main(String args[]){
new B().m1();
}
class A{
public void m1(){ }
}
class B extends A{
protected void m1(){ }
}

```

OOPS372.java

```

class OOPS372{
public static void main(String args[]){
new B().m1();
}
class A{
public void m1(){ }
}
class B extends A{
public void m1(){ }
}

```

OOPS373.java

```

class OOPS373{
public static void main(String args[]){
B obj=new B();
obj.m1();
}
class A{
void m1(){
System.out.println("A->m1()");
}
}
class B extends A{
void m1(){
System.out.println("B->m1()");
}
}

```

OOPS374.java

```

class OOPS374{
public static void main(String args[]){
B obj=new B();
obj.m1();
}
class A{

```

```

void m1(){
System.out.println("A->m1()");
}
class B extends A{ void m1()
{
System.out.println("B->m1()");
m1();
}
}

```

OOPS375.java

```

class OOPS375{
    public static void main(String args[]){
        B obj=new B();
        obj.m1();
    }
    class A{
        void m1(){
            System.out.println("A->m1()");
        }
    }
    class B extends A{
        void m1(){
            System.out.println("B->m1()");
            super.m1();
        }
    }
}

```

OOPS376.java

```

class OOPS376{
    public static void main(String args[]){
        B obj=new B();
        obj.m1();
    }
    class A{
        static void m1(){
            System.out.println("A->m1()");
        }
    }
    class B extends A{
        static void m1()
        {
            System.out.println("B->m1()");
        }
    }
}

```

Hello.java

```

package com.p1;
public class Hello{
    protected void display ( ) { }
}

```

OOPS377.java

```

package com.p2;
import com.p1.Hello;
class Xyz extends Hello{ }
class Hai extends Hello{
    void show(){
        Xyz ref=new Xyz();
        ref.display();
    }
}

class OOPS377{
    public static void main(String args[]){
        Hai hai=new Hai();
        hai.show();
    }
}

```

OOPS378.java

```

package com.p2;
import com.p1.Hello;
class Xyz extends Hello{
    protected void display(){ }
}
class Hai extends Hello{
    void show(){
        Xyz ref=new Xyz();
        ref.display();
    }
}

class OOPS378{
    public static void main(String args[]){
        Hai hai=new Hai();
        hai.show();
    }
}

```

Hello1.java

```

package com.p1;
public class Hello1{
    void display ( ) { }
}

```

OOPS379.java

```

package com.p2;
import com.p1.Hello1;
class Hai extends Hello1{
    void display(){
        System.out.println("Hai->void display()");
    }
}

class OOPS379{
    public static void main(String args[]){
        Hai hai=new Hai();
        hai.display();
    }
}

```

OOPS380.java

```

package com.p2;
import com.p1.Hello1;
class Hai extends Hello1{
    int display(){
        System.out.println("Hai->int display()");
        return 0;
    }
}

class OOPS380{
    public static void main(String args[]){
        Hai hai=new Hai();
        hai.display();
    }
}

```

Program output /error and details:-

Prog No.	Output	Error and Details
OOPS353	Hello->m1() Hai->m2() Hai->m3()	Refer page 127 for details.
OOPS354	A->show() B->SHOW()	Here, We are using different method argument in subclass then it is method overloading not overriding.
OOPS355	A->show(int) B->SHOW(String)	Subclass method parameters (type, order and number) must be same as super class method parameter.
OOPS356	Compilation Error	[Error : The return type is incompatible with A. show()] Refer page 127 point 3 for details.
OOPS357	Compilation Error	[Error: The return type is incompatible with Hello.m1()] Refer page 127 point 3 for details.
OOPS358	Blank	We are not using any sop statement so it give blank output.
OOPS359	Blank	same""
OOPS360	Compilation Error	[Error: class B overrides final method m1 ()] Final method of super class cannot be overridden in sub class.
OOPS361	Blank	Non final super class can be overridden as final in sub class.
OOPS362	Compilation Error	[Error : This static method cannot hide the instance method from A] Refer page 128 point 5 for details.
OOPS363	Compilation Error	[Error : Cannot make a static reference to the non-static method m1() from the type B] Refer page 128 point 6 for details.
OOPS364	Compilation Error	[Error : Cannot make a static reference to the non-static method m1() from the type B] same""
OOPS365	Blank	Refer page 128 point 15 and 16 for details.
OOPS366	Blank	Refer page 128 point 21 for details.

Prog No.	Output	Error and Details
OOPS367	Compilation Error	[Error : The method m1() from the type B is not visible] Sub class of private method is not visible outside the class.
OOPS368	Blank	Subclass method access modifier must be same or higher then super class method access modifier. Here, super class method is default type and sub class method is default, protected and public type so it is accessible. For more details Refer page 127 point 4.
OOPS369	Compilation Error	[Error : The method m1() from the type B is not visible] Refer page 127 point 4 for details
OOPS370	Blank	Refer page 127 point 4 for details
OOPS371	Compilation Error	[Error : Cannot reduce the visibility of the inherited method from A] Refer page 127 point 4 for details
OOPS372	Blank	Refer page 127 point 4 for details
OOPS373	B->m1()	When the method is overridden in the sub class and if you are using sub class object to invoke the method then implementation from sub class will be invoked.
OOPS374	B->m1() B->m1() StackOverflowError	When the method is overridden in the sub class and if you are using sub class object to invoke the method then implementation from sub class will be invoked. But again we are calling m1 method so it will execute again and again.
OOPS375	B->m1() A->m1()	When the method is overridden in the sub class and if you are using sub class object to invoke the method then implementation from sub class will be invoked. But again we are calling m1 method with super reference so super class implementation will also invoked.
OOPS376	B->m1()	Refer page 128 point 13 for details.
OOPS377	Blank	Refer page 128 point 22 and 23 for details.
OOPS378	Blank	same"
OOPS379	Hai->void display()	same"
OOPS380	Hai->int display()	same"

Write your note here.....